

Quantum-Resistant Blockchain Framework for Secure AI Model Distribution and Provenance

A Hybrid Ledger Architecture Combining NIST FIPS 204 (ML-DSA), Hyperledger Fabric, Ethereum Anchoring and IPFS Content Addressing

Authors	Registration No.	Department	Submission
Madhusudhan Mahathi Shri G	25BCE1176 25BCE5747	Computer Science & Engineering	Digital Assignment 2 (DA2) AY 2025–2026

Abstract — AI model distribution pipelines authenticate artefacts using RSA and ECDSA signatures whose security collapses under Shor’s algorithm. Because a blockchain preserves historical signatures permanently, a future Cryptographically Relevant Quantum Computer (CRQC) can retroactively forge provenance records that were considered valid when written. This report specifies and implements a quantum-resistant provenance framework built on NIST FIPS 204 ML-DSA-65 signatures, a dual-layer Hyperledger Fabric + Ethereum ledger, and IPFS content-addressed storage. Three design contributions distinguish it from generic file-provenance systems: on-chain signature *enforcement* inside Fabric chaincode rather than passive key storage; *per-tensor Merkle hashing* that localises tampering to a specific layer and enables streaming verification of multi-gigabyte models; and a *composite Ed25519 + ML-DSA signature* scheme for migration-period assurance. The report also states, rather than conceals, the framework’s residual quantum exposure: the Ethereum anchoring transaction is itself secp256k1-signed.

1. Introduction

1.1 Background and Motivation

Machine learning models are now distributed artefacts. Foundation models, fine-tuned derivatives and edge-deployed vision networks move continuously between organisations, cloud regions and devices, traversing networks that none of the participating parties fully controls. Federated learning and multi-tenant MLOps pipelines extend this further, requiring that weight updates and hyperparameter configurations be exchanged across mutually distrusting boundaries.

Distributed ledger technology answers the trust half of this problem. An append-only ledger of model hashes, authorship metadata and version history provides audit-ready provenance across a model’s lifecycle without a central registry operator. But the ledger inherits its security from the signature scheme underneath it, and every production blockchain today authenticates transactions with ECDSA over secp256k1 or with RSA-backed X.509 identities. Both reduce to integer factorisation or the elliptic-curve discrete logarithm problem, and Shor demonstrated in 1994 that a sufficiently large quantum computer solves both in polynomial time [5].

1.2 The Immutability Paradox

Blockchain’s defining property becomes its defining liability under a quantum threat model. A creator who signs a model hash with ECDSA today writes that signature, and the public key that verifies it, permanently onto thousands of replicated nodes. When a CRQC becomes available — current expert estimates cluster in the 2030–2040 range — an adversary retrieves that historical public key from the ledger, recovers the private key, and forges signatures for poisoned or backdoored models that appear to predate the compromise. The chain cannot rewrite the affected history without breaking consensus, so the entire trust lineage is invalidated retroactively.

Terminology note. This is distinct from the Harvest-Now-Decrypt-Later (HNDL) attack, with which it is frequently conflated. HNDL is a *confidentiality* attack: an adversary stores ciphertext today and decrypts it once the key-exchange primitive falls. Signatures are not harvested and decrypted; they are *retroactively forged*. The two require different countermeasures — ML-KEM (FIPS 203) for the former, ML-DSA (FIPS 204) for the latter — and this framework addresses both, in Sections 3.2 and 3.5 respectively.

1.3 Problem Statement

Current AI model distribution exhibits three systemic weaknesses:

1. Cryptographic obsolescence. Authorship and integrity rest on RSA/ECDSA signatures that are recoverable by quantum key extraction, and the permanence of ledger storage converts a future break into a retroactive one.
2. Centralised trust roots. Model registries depend on centralised hubs and PKI certificate authorities that can be breached, compelled or misconfigured. Removing the CA, however, does not remove the underlying problem of binding a public key to a real-world vendor identity — a gap most decentralised proposals leave open.

3. Absent integrated quantum-safe provenance. Post-quantum signature schemes, blockchain provenance systems and MLOps lineage tooling each exist in mature form, but no published framework composes all three around the specific constraints of AI artefacts: multi-gigabyte payloads, frequent derivative works (fine-tunes, LoRA adapters), and deserialisation-based supply-chain attacks.

1.4 Objectives

- **O1 — Integrated design.** Compose FIPS 204 ML-DSA-65, a Fabric/Ethereum dual-layer ledger and IPFS content addressing into a deployable architecture with a working reference implementation.
- **O2 — Enforced, not advertised, quantum safety.** Verify post-quantum signatures as an endorsement condition inside Fabric chaincode, so an invalid signature cannot be committed — rather than merely storing the public key on-chain for optional client-side checking.
- **O3 — AI-specific tamper localisation.** Replace flat whole-file hashing with a per-tensor Merkle tree, enabling single-layer tamper identification, streaming verification without full download, and cheap delta-signing of fine-tuned derivatives.
- **O4 — Honest residual-risk accounting.** Document the boundary of the quantum guarantee, including the secp256k1 dependency of the Ethereum anchor transaction and the non-persistence of unpinned IPFS content.

1.5 Scope

In scope: Fabric v2.5 network provisioning; Go chaincode with embedded ML-DSA verification; Solidity anchor contract deployed to a local Hardhat chain and the Sepolia testnet; React + ethers.js frontend; Kubo IPFS node; per-tensor Merkle hasher; unit and fault-injection test suites; gas and latency benchmarks.

Out of scope: Ethereum mainnet deployment with real value; FPGA/ASIC acceleration of lattice polynomial arithmetic; physical quantum hardware benchmarking; zero-knowledge proof circuits for on-chain ML-DSA verification (identified as future work in Section 3.10).

2. Literature Survey

2.1 Blockchain for AI Governance and Ownership

Battah et al. [9] use smart contracts and NFTs to represent ownership, licensing and access rights over trained models, demonstrating that contract state machines are adequate for expressing model governance. Ural and Yoshigoe [8] survey blockchain-enhanced machine learning broadly, covering incentive design, decentralised training markets and provenance ledgers. Both lines of work assume classical signature schemes throughout: security is analysed against dishonest participants, never against a computationally quantum adversary, so their guarantees expire at the same moment secp256k1 does.

2.2 Provenance and Lineage in MLOps

Mitchell et al. [10] introduced Model Cards as structured documentation of a model's intended use, training data and evaluated performance, and the pattern is now standard practice. Kreuzberger et al. [15] formalise the MLOps architecture within which artefact tracking tools such as MLflow and Kubeflow operate. These provide excellent operational lineage but store it in mutable centralised databases: an administrator or a compromised service account can rewrite history without detection, and none of them produce a cryptographic non-repudiation guarantee that survives insider action.

2.3 Federated Learning Security

Kairouz et al. [18] catalogue the open problems in federated learning, with the literature concentrating on privacy-preserving aggregation — differential privacy, secure multi-party computation and homomorphic techniques applied to gradient updates. The threat model is overwhelmingly training-phase. Once a global model has been aggregated, downstream distribution to edge devices is treated as an implementation detail and is generally assumed secure, which is precisely the phase this framework addresses.

2.4 Post-Quantum Cryptography and Its Integration into Ledgers

NIST finalised FIPS 204 in August 2024, standardising ML-DSA — the standardised form of CRYSTALS-Dilithium [3] — as the primary general-purpose post-quantum signature algorithm, alongside FIPS 203 (ML-KEM) for key encapsulation [1], [2]. The scheme's security reduces to Module-LWE and Module-SIS, hard lattice problems for which no polynomial-time quantum algorithm is known [14], and its signing procedure uses the Fiat-Shamir-with-aborts rejection sampling technique derived from Lyubashevsky's trapdoor-free lattice signatures [4]. NIST SP 800-208 [13] is sometimes cited in

this context but governs stateful hash-based schemes (LMS and XMSS) and is not applicable to ML-DSA; the correct compliance reference for this framework is FIPS 204.

Fernández-Caramès and Fraga-Lamas [7] provide the closest prior work, reviewing post-quantum cryptography specifically in a blockchain setting and cataloguing the signature-size and throughput penalties that lattice schemes impose on transaction formats. Their review is, however, cryptocurrency-centric and does not consider heavy off-chain payloads, MLOps workflows or AI-specific artefact structure.

2.5 Comparative Analysis Matrix

Reference	Core Contribution	Ledger	PQC	Principal Limitation
Ducas et al. [3]; NIST FIPS 204 [1]	ML-DSA lattice signature scheme and its standardisation	None	Yes	Pure primitive; no distributed state layer, no artefact-management context
Lyubashevsky [4]	Trapdoor-free lattice signatures; rejection sampling	None	Yes	Theoretical foundation only
Benet [6]	IPFS content-addressed P2P file system	DHT / libp2p	No	Storage only; no governance, no persistence guarantee without pinning
Buterin [12]	Programmable public ledger	Ethereum	No (secp256k1)	ECDSA transaction signing; gas cost prohibits large payloads
Hyperledger [11]	Permissioned ledger with MSP-based RBAC	Fabric v2.5	No (X.509 RSA)	Classical PKI trust root; no public auditability
Battah et al. [9]	NFT-based model ownership and access control	Ethereum	No	Quantum-vulnerable ownership records; no integrity binding to weights
Ural & Yoshigoe [8]	Survey of blockchain-enhanced ML	Mixed	No	Classical threat model throughout
Mitchell et al. [10]	Model Cards for structured reporting	None	No	Documentation convention; no cryptographic binding
Fernández-Caramès & Fraga-Lamas [7]	Review of post-quantum blockchain cryptography	Mixed	Yes	Cryptocurrency-focused; no large-payload or AI artefact treatment
This work	Enforced ML-DSA verification + per-tensor Merkle provenance	Hybrid (Fabric + Eth)	Yes (ML-DSA-65)	Anchor transaction remains secp256k1-signed (Section 3.10)

Table 2.1 — Comparative positioning against representative prior work.

2.6 Identified Research Gap

The literature partitions cleanly along three axes that no single work spans. Blockchain-for-AI research assumes classical cryptography. Post-quantum cryptography research treats signatures as network protocol components, benchmarking TLS handshakes rather than ledger state. MLOps provenance tooling assumes a trusted central store. The gap this work addresses is therefore not “use PQC on a blockchain” — which [7] already anticipates — but the harder composition problem: enforcing post-quantum authorship inside a permissioned ledger’s endorsement policy, at a payload scale where the artefact cannot go on-chain, with verification granularity matched to the internal structure of neural network files.

3. Methodology

3.1 System Architecture

The framework comprises six components arranged in a publish-then-verify pipeline. Fabric performs high-frequency MLOps state transitions with signature enforcement and no gas cost; Ethereum receives periodic Merkle-root commitments that provide public, tamper-evident proof of existence without exposing proprietary metadata.

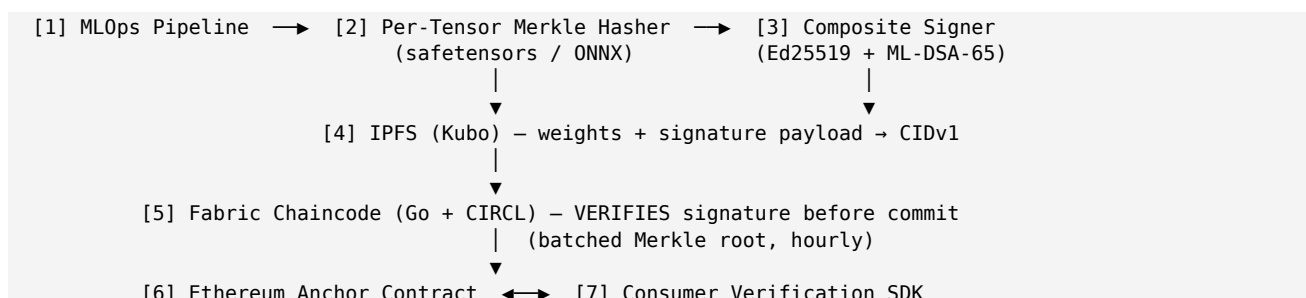


Figure 3.1 — End-to-end architecture. Verification occurs at component [5], not only at [7].

3.2 Cryptographic Design

The framework selects ML-DSA-65 (NIST Security Category 3, comparable to AES-192 key search, i.e. approximately 2^{192} classical effort). Parameter sizes are taken from FIPS 204 as standardised, which differ from the widely reproduced Round-3 Dilithium figures; the discrepancy is noted because several published surveys still quote the older values.

Parameter Set	NIST Level	Public Key (B)	Private Key (B)	Signature (B)	Round-3 signature (B)
ML-DSA-44	Category 2	1,312	2,560	2,420	2,420
ML-DSA-65 (selected)	Category 3	1,952	4,032	3,309	3,293
ML-DSA-87	Category 5	2,592	4,896	4,627	4,595

Table 3.1 — FIPS 204 parameter sets. The final column shows the superseded Round-3 Dilithium values for comparison.

Composite signatures. Following IETF migration guidance, the signer produces a concatenated Ed25519 || ML-DSA-65 signature and the verifier requires both to pass. This hedges against an unforeseen cryptanalytic result against module lattices during the transition period, at a cost of 64 additional bytes. The signed message is a domain-separated context string,

$$\mu = \text{SHAKE256}(\text{"QRB-v1"} \parallel \text{chainID} \parallel \text{modelId} \parallel \text{version} \parallel \text{merkleRoot} \parallel \text{pkHash})$$

which binds every signature to a single chain, model identifier and version, preventing cross-version and cross-chain replay.

Confidentiality. Where model weights are proprietary, the payload is encrypted with AES-256-GCM under a content encryption key wrapped using ML-KEM-768 (FIPS 203) [2] rather than ECDH. Without this, the HNDL vector identified in Section 1.2 would remain unmitigated regardless of signature strength.

3.3 Per-Tensor Merkle Hashing

A flat SHA-256 over the serialised file detects tampering but cannot say where it occurred, requires the full multi-gigabyte download before any verification is possible, and forces a complete re-signature for a fine-tune that modifies a handful of adapter matrices. The framework instead parses the artefact as a safetensors container, hashes each named tensor independently, sorts by tensor name for determinism, and constructs a binary Merkle tree whose root is the signed value:

```
leaf_i    = SHA-256( name_i || dtype_i || shape_i || tensor_bytes_i )
root      = MerkleRoot( sort_by_name( leaf_0 ... leaf_n ) )
H_meta    = SHA-256( model_card || dataset_hash || eval_metrics )
signed_μ  = SHAKE256( context || root || H_meta )
```

This yields three properties a flat hash cannot. Tamper localisation: a failing inclusion proof names the specific layer that was modified. Streaming verification: a consumer validates each tensor as it arrives and aborts on the first mismatch. Delta provenance: a LoRA adapter re-signs only the changed leaves and records a pointer to its parent root, producing a lineage DAG rather than a flat list of unrelated artefacts.

Deserialisation defence. The consumer SDK verifies before it loads. The dominant real-world AI supply-chain attack today is not quantum but arbitrary code execution through pickle deserialisation in *torch.load*; a signature checked after loading is worthless. The SDK therefore rejects pickle-backed formats outright and accepts only safetensors and ONNX.

3.4 Storage Layer

Model binaries are pinned to a Kubo IPFS cluster, which chunks the file into a UnixFS Merkle DAG and returns a CIDv1. It is worth stating the content identifier precisely, since it is commonly misreported:

```
CID = CIDv1( codec = dag-pb, multihash = SHA-256( UnixFS DAG root ) )
```

For any file exceeding the 256 KiB default chunk size, this root differs from the SHA-256 digest of the file bytes, so the Merkle root of Section 3.3 is computed and recorded separately. IPFS content addressing guarantees integrity but not persistence: unpinned blocks are garbage-collected. The reference deployment therefore maintains three pinning nodes plus a Filecoin storage deal for artefacts under long-term retention policy, and the residual availability risk is rated MEDIUM in Table 3.3.

3.5 Smart Contract and Chaincode Design

Responsibility is split deliberately. Fabric chaincode performs full ML-DSA verification during endorsement, using the Cloudflare CIRCL library [16], which provides a native Go ML-DSA implementation and therefore avoids cgo bindings. Because Fabric has no gas metering, a 3,309-byte signature verification costing roughly 0.3 ms is entirely affordable as an endorsement condition — an invalid signature is rejected before it can be committed. Ethereum, where the same verification would be prohibitively expensive in EVM opcodes, stores only commitments.

The anchor contract was revised from the initial design to remove three defects: a 1,952-byte on-chain public key costing roughly 1.2 M gas in storage writes, an indexed string event parameter (which Solidity stores as a keccak hash, making the model ID unreadable from logs), and an existence check that blocked legitimate version updates.

```

struct Anchor {
    bytes32 merkleRoot;      // per-tensor Merkle root (Section 3.3)
    bytes32 cidDigest;      // CIDv1 multihash digest, not a string
    bytes32 pkHash;         // keccak256(ML-DSA pk); full key lives on IPFS
    bytes32 parentRoot;     // lineage pointer – 0x0 for a base model
    uint64  version;
    uint64  timestamp;
    bool    revoked;
}
mapping(bytes32 => mapping(uint64 => Anchor)) private registry; // id → version

event ModelAnchored(bytes32 indexed modelIdHash, string modelId,
    uint64 version, bytes32 merkleRoot, bytes32 cidDigest);
event ModelRevoked(bytes32 indexed modelIdHash, uint64 version, string reason);

function anchorModel(string calldata modelId, uint64 version, bytes32 merkleRoot,
    bytes32 cidDigest, bytes32 pkHash, bytes32 parentRoot)
    external onlyPublisher {
    bytes32 k = keccak256(bytes(modelId));
    require(registry[k][version].merkleRoot == 0, "version already anchored");
    require(version == 1 || registry[k][version-1].merkleRoot != 0, "gap in lineage");
    /* ... store, emit ... */
}

```

Figure 3.2 — Revised anchor contract core. Versions are additive, so lineage is preserved rather than blocked.

Publisher management is placed behind a 2-of-3 Gnosis Safe multisignature wallet, and revocation is a first-class operation: a compromised signing key can be retired and the affected versions flagged without rewriting history.

3.6 Network Setup and Frontend Integration

Layer	Configuration	Purpose / Verification Artefact
Fabric	v2.5 LTS, 2 organisations × 2 peers, 3-node Raft orderer, channel "mlops", Go chaincode with CIRCL	Signature enforcement; endorsement policy AND(Org1MSP.peer, Org2MSP.peer). Evidence: peer logs showing endorsement rejection on a corrupted signature
Ethereum	Hardhat local node (chainId 31337) for development; Sepolia testnet for public anchoring; Solidity 0.8.24	Public proof of existence. Evidence: deployed contract address and anchoring transaction hashes
IPFS	Kubo v0.28, 3-node private swarm, Pinata fallback pinning	Payload storage. Evidence: CIDv1 values and pin status output
Crypto	liboqs / python-oqs for the signer CLI [17]; CIRCL for chaincode; PyNaCl for Ed25519	Keypair generation and composite signing
Frontend	React 18 + ethers.js v6 + Fabric Gateway SDK; MetaMask for anchor transactions	Upload → hash → sign → pin → submit → verify workflow with live status
Consumer SDK	Python package: fetch CID, stream-verify tensors, check ML-DSA signature, confirm anchor inclusion proof	Refuses to return a model object if any check fails

Table 3.2 — Deployment environment and the evidence each layer produces for the functional demonstration.

The frontend exposes four flows: model registration, lineage browsing (rendering the parent-pointer DAG), verification of a downloaded artefact with per-tensor pass/fail display, and publisher/revocation administration. Fabric interactions use the Gateway SDK over gRPC; Ethereum anchoring is signed in the browser through MetaMask.

3.7 Evaluation Methodology

The following measurements constitute the results section of the functional demonstration. Each is repeated thirty times and reported as a median with interquartile range.

- Signing and verification latency for ML-DSA-65 versus ECDSA-secp256k1 and Ed25519, measured separately from artefact I/O.
- Per-tensor Merkle construction throughput against model size (10 MB, 100 MB, 1 GB, 7 GB), compared with flat SHA-256, to quantify the overhead of tamper localisation.
- Gas consumption of anchorModel() and revokeModel(), and the amortised cost per model when N transitions are batched into one Merkle root.

- Fabric endorsement latency and throughput (TPS) with signature verification enabled versus disabled, isolating the cost of the enforcement decision.
- Fault injection: single-bit flips in a randomly selected tensor, tensor reordering, CID substitution, replay of a prior version, and registration by an unauthorised publisher. Expected outcome is detection in every case, with the per-tensor test additionally naming the affected layer.

3.8 Threat Model and Residual Risk

STRIDE Class	Threat	Control	Residual
Spoofing	Adversary publishes a malicious model under a trusted vendor identity	Composite Ed25519 + ML-DSA-65 verified during Fabric endorsement	LOW
Tampering	Weight modification in transit or at an IPFS gateway	Per-tensor Merkle root; streaming verification with layer-level localisation	LOW
Repudiation	Vendor denies authoring a model that later failed	Non-repudiable ML-DSA signature anchored to Ethereum; note block.timestamp is miner-influenceable by seconds	LOW
Information disclosure	Leakage of proprietary architecture through public metadata	Weights encrypted under ML-KEM-768-wrapped AES-256-GCM; metadata confined to Fabric private data collections	LOW
Denial of service	Unpinned IPFS content garbage-collected; registry spam	Triple pinning plus Filecoin deal; onlyPublisher modifier and Fabric MSP gatekeeping	MEDIUM
Elevation of privilege	CRQC recovers the publisher's secp256k1 key and anchors forged records	2-of-3 multisig on publisher management; Fabric-side ML-DSA check still fails, so forged anchors are detectable but not preventable	MEDIUM

Table 3.3 — STRIDE assessment. Two risks are deliberately rated MEDIUM rather than mitigated to LOW.

Stated limitation. The Ethereum anchoring transaction is authorised by `msg.sender` and therefore signed with `secp256k1`. A quantum adversary who recovers a publisher's externally owned account key can write anchors, even though they cannot forge the ML-DSA signature those anchors reference — meaning forged anchors fail verification but can still be committed. Closing this gap requires an ERC-4337 smart account with an ML-DSA verifier module, or a zk-SNARK proving correct off-chain lattice verification so the EVM checks only a succinct proof. Both are identified as the primary direction for continued work; neither is claimed as implemented here.

3.9 Implementation Schedule

Week	Phase	Deliverable
1	Survey, threat modelling, parameter selection	Literature matrix; STRIDE table; ML-DSA-65 justification
2	Fabric network and Hardhat environment provisioning	Running 2-org channel; local chain with funded accounts
3	Chaincode and Solidity contract development	CIRCL-based verification chaincode; anchor contract with passing unit tests
4	Signer CLI and per-tensor Merkle hasher	Composite signing tool; Merkle hasher validated on a 1 GB model
5	IPFS pipeline, frontend integration, end-to-end run	React UI completing upload→anchor→verify; fault-injection results
6	Benchmarking, Sepolia deployment, documentation	Gas and latency tables; public contract address; final report

References

- [1] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard," FIPS PUB 204, Gaithersburg, MD, USA, Aug. 2024.
- [2] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS PUB 203, Gaithersburg, MD, USA, Aug. 2024.
- [3] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme," IACR Transactions on Cryptographic Hardware and Embedded Systems, vol. 2018, no. 1, pp. 238–268, 2018.
- [4] V. Lyubashevsky, "Lattice Signatures Without Trapdoors," in Advances in Cryptology – EUROCRYPT 2012, LNCS vol. 7237, Springer, 2012, pp. 738–755.
- [5] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS), IEEE, 1994, pp. 124–134.

- [6] J. Benet, "IPFS — Content Addressed, Versioned, P2P File System," arXiv preprint arXiv:1407.3561, 2014.
- [7] T. M. Fernández-Caramès and P. Fraga-Lamas, "Towards Post-Quantum Blockchain: A Review on Blockchain Cryptography Resistant to Quantum Computing Attacks," *IEEE Access*, vol. 8, pp. 21091–21116, 2020.
- [8] O. Ural and K. Yoshigoe, "Survey on Blockchain-Enhanced Machine Learning," *IEEE Access*, vol. 11, pp. 145331–145362, 2023.
- [9] A. Battah, M. Madine, I. Yaqoob, K. Salah, H. Al-Hammadi, and R. Jayaraman, "Blockchain and NFTs for Trusted Ownership, Trading, and Access of AI Models," *IEEE Access*, vol. 10, pp. 112230–112249, 2022.
- [10] M. Mitchell et al., "Model Cards for Model Reporting," in *Proc. Conference on Fairness, Accountability, and Transparency (FAT*)*, ACM, 2019, pp. 220–229.
- [11] Hyperledger Foundation, "Hyperledger Fabric v2.5 LTS Documentation," 2023. [Online]. Available: <https://hyperledger-fabric.readthedocs.io>
- [12] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," *Ethereum White Paper*, 2014. [Online]. Available: <https://ethereum.org/en/whitepaper/>
- [13] D. Cooper, D. Apon, Q. Dang, M. Davidson, M. Dworkin, and C. Miller, "Recommendation for Stateful Hash-Based Signature Schemes," *NIST Special Publication 800-208*, Oct. 2020.
- [14] C. Peikert, "A Decade of Lattice Cryptography," *Foundations and Trends in Theoretical Computer Science*, vol. 10, no. 4, pp. 283–424, 2016.
- [15] D. Kreuzberger, N. Kühn, and S. Hirschl, "Machine Learning Operations (MLOps): Overview, Definition, and Architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023.
- [16] Cloudflare, "CIRCL: Cloudflare Interoperable Reusable Cryptographic Library," 2024. [Online]. Available: <https://github.com/cloudflare/circl>
- [17] D. Stebila and M. Mosca, "Post-Quantum Key Exchange for the Internet and the Open Quantum Safe Project," in *Selected Areas in Cryptography (SAC)*, LNCS vol. 10532, Springer, 2017, pp. 14–37.
- [18] P. Kairouz et al., "Advances and Open Problems in Federated Learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.